



Repetitions

repetitions



Easy

General Skills

picoCTF 2023

base64

AUTHOR: THEONESTE BYAGUTANGAZA

Description

Can you make sense of this file?

Download the file [here](#).

Hints ?

1

Multiple decoding is always good.

39,855 users solved



90%

Liked



🚩 picoCTF{FLAG}

Submit
Flag

Author: The Analyst: Hyposelenia

Challenge: Repetitions

Category: General Skills

Date: 01/16/26



I. Objective

To retrieve the hidden flag by repeatedly decoding the contents of the provided file until readable text is obtained.

II. Background

Base64 is a common **encoding scheme** used to represent binary data in a text format. It is widely used in programming, file transfers, and CTF challenges because it safely represents data using readable characters.

In CTFs, Base64 is often **nested multiple times**, meaning the decoded output is still Base64-encoded and must be decoded again. This challenge demonstrates that concept clearly.

III. Tool Used

- **Oracle VirtualBox (Kali Linux)** – Used as the Linux environment
- **Linux Terminal** – Used to execute decoding commands
- **base64 utility** – Built-in Linux tool for decoding Base64 text

IV. Methodology

1. Downloaded the provided file and navigated to its directory.
2. Displayed the file contents using the cat command.
3. Observed that the text consisted of characters commonly associated with Base64 encoding.
4. Used **Ctrl + Shift + C** to copy and **Ctrl + Shift + V** to paste within the Linux terminal for efficiency.



5. Decoded the file using the command:

```
echo "<encoded_text>" | base64 -d
```

6. Noticed that the output was **still Base64-encoded**, so the same command was repeated.

7. Continued decoding multiple times—similar to opening a **Russian nesting doll (Matryoshka)** where each layer contains another encoded layer.

8. Repeated this process until the final output revealed a readable picoCTF flag.

V. Result

picoCTF{base64_n3st3d_dic0d!n8_d0wnl04d3d_de523f49}

```
File Actions Edit View Help
(kali@kali) ~/media/sf_Downloaded_Cybersecfiles
$ ls
enc_flag  Screenshot_2026-01-15_20_32_13.png
(kali@kali) ~/media/sf_Downloaded_Cybersecfiles
$ cat enc_flag
VmpGU1EyR1LWgXtYmxKVVYwZFNWbGxyV2lGV1JteDBUbFpPYWxkdFVsaFpWVUxWVZaSlZWnnVh
RnRXZwablDmmtSMk5yTLZWAp1VpUvVm10d1VMZFdvZ2RyYlZaWZtNVdVZ3BpU0VkeLdWUkNk
MLZxVlhowGJYQk9YbFJXU0ZkcVRuTLdaM0JZWpGS2VWwkdSGRXCk1swmpW3hhVm1KRK5XOVVW
VkpEVGxaYVdFMhShBHW0VWVZGmFWMDHV2tkYVNHU1ZDazFyY0ZkYwJGwLhZVlpLU0dWR1Zs
aGKYLRreLZERldUMkpZUmxWTLJYTKx0Zz09Cg==
(kali@kali) ~/media/sf_Downloaded_Cybersecfiles
$ echo "VmpGU1EyR1LWgXtYmxKVVYwZFNWbGxyV2lGV1JteDBUbFpPYWxkdFVsaFpWVUxWVZaSlZWnnVhRnRXZwablDmmtSMk5yTLZWAp1VpUvVm10d1VMZFdvZ2RyYlZaWZtNVdVZ3BpU0VkeLdWUkNkMLZxVlhowGJYQk9YbFJXU0ZkcVRuTLdaM0JZWpGS2VWwkdSGRXCk1swmpW3hhVm1KRK5XOVVWVkpEVGxaYVdFMhShBHW0VWVZGmFWMDHV2tkYVNHU1ZDazFyY0ZkYwJGwLhZVlpLU0dWR1ZsaGKYLRreLZERldUMkpZUmxWTLJYTKx0Zz09Cg==" | base64 -d
VjFSQ2EyTXlSb1JUV0dSVllrmmFWmX0TLZ0a1JtUlhZVWU1YVZKVVZuaFdwZWVZK2NzNVVX
bU2TVmtwJVdWUkdibVZKXm5WUgplSE3ZwVRCd2VWVXhXbOU1RWSfdqTnNWZ3BYUjFkeVZGZhdW
MLZzVWxaVmJFNW9UVVJDTLZaWE1XRlpVWEJUVFZaV0SgWkdSGRVCk1rcFduBfZXVZKSgVFVlh1
bTkzVDFWT2J3QlVNRXNLCg==
(kali@kali) ~/media/sf_Downloaded_Cybersecfiles
$ echo "VjFSQ2EyTXlSb1JUV0dSVllrmmFWmX0TLZ0a1JtUlhZVWU1YVZKVVZuaFdwZWVZK2NzNVVXbU2TVmtwJVdWUkdibVZKXm5WUgplSE3ZwVRCd2VWVXhXbOU1RWSfdqTnNWZ3BYUjFkeVZGZhdWMLZzVWxaVmJFNW9UVVJDTLZaWE1XRlpVWEJUVFZaV0SgWkdSGRVCk1rcFduBfZXVZKSgVFVlh1bTkzVDFWT2J3QlVNRXNLCg==" | base64 -d
V1RCa2MyRnRtWGRVYkZaVFltNVNjRmRYU5aVJUVnhWVzFhYVdGckSUmF5VkpQWVRGbmVWnVR
bHBsYTBweVUxmpNRTVHmJNsVgpXR1JyVdW2VsU1ZVbES0TURCNVZXMFZUX08TTVZWNFZGZhdU
MkpWTLVWavJHeEVXbm93TlVob1BUMESk
(kali@kali) ~/media/sf_Downloaded_Cybersecfiles
$ echo "V1RCa2MyRnRtWGRVYkZaVFltNVNjRmRYU5aVJUVnhWVzFhYVdGckSUmF5VkpQWVRGbmVWnVRbHBsYTBweVUxmpNRTVHmJNsVgpXR1JyVdW2VsU1ZVbES0TURCNVZXMFZUX08TTVZWNFZGZhdUMkpWTLVWavJHeEVXbm93TlVob1BUMESk" | base64 -d
WTBkczF1SXdubFZTm5ScFdwE91RTxvWlaaWFnTzARV3PYTFneVvuQlpla0pyU1ZjME5G23lV
WGRtWpWe1RVU1NhMDBSVW1aYQpSMVAVFdwT2JUVU1RGxEmnowOUNPT0k
(kali@kali) ~/media/sf_Downloaded_Cybersecfiles
$ echo "WTBkczF1SXdubFZTm5ScFdwE91RTxvWlaaWFnTzARV3PYTFneVvuQlpla0pyU1ZjME5G23lVWGRtWpWe1RVU1NhMDBSVW1aYQpSMVAVFdwT2JUVU1RGxEmnowOUNPT0kYDdsam1wTLV5bnRpVH0bESqumZiaK56ZER0a1gyUm8Zek3rSVc0NFgyUXdkMjvTURSaa00ymZaR1UxTWpObUSEB0D0Zz09Cg==" | base64 -d
YDdsam1wTLV5bnRpVH0bESqumZiaK56ZER0a1gyUm8Zek3rSVc0NFgyUXdkMjvTURSaa00ymZa
R1UxTWpObUSEB0D0Zz09Cg==
(kali@kali) ~/media/sf_Downloaded_Cybersecfiles
$ echo "YDdsam1wTLV5bnRpVH0bESqumZiaK56ZER0a1gyUm8Zek3rSVc0NFgyUXdkMjvTURSaa00ymZaR1UxTWpObUSEB0D0Zz09Cg==" | base64 -d
cG1jb0NURnt1YXNlNjRfYjNz0d0NkX2RyZk1W44X2Qwd25sMDRkM2RfZGU1MjNmNDl9Cg==
(kali@kali) ~/media/sf_Downloaded_Cybersecfiles
$ echo "cG1jb0NURnt1YXNlNjRfYjNz0d0NkX2RyZk1W44X2Qwd25sMDRkM2RfZGU1MjNmNDl9Cg==" | base64 -d
picoCTF{base64_n3st3d_dic0d!n8_d0wnl04d3d_de523f49}
(kali@kali) ~/media/sf_Downloaded_Cybersecfiles
```



VI. Explanation

Base64 is an encoding method that converts data into a set of 64 readable characters:

- Uppercase letters (A–Z)
- Lowercase letters (a–z)
- Numbers (0–9)
- Symbols (+ and /)
- Often ends with padding characters like = or ==

You can usually **identify Base64** when:

- The text contains only letters, numbers, +, /, and =
- The output looks “random” but decodes cleanly without errors
- Decoding produces another similarly formatted string

This challenge relies on **repetition** rather than complexity. Each decoded layer reveals another Base64 string, requiring the same decoding process again and again. The challenge teaches persistence and pattern recognition rather than advanced cryptography.

VII. Conclusion

The **Repetitions** challenge demonstrates how Base64 encoding can be layered multiple times to obscure data. By recognizing Base64 patterns and repeatedly applying the correct decoding method, the **hidden flag can be successfully uncovered**. This challenge reinforces the importance of patience, observation, and familiarity with basic encoding techniques in cybersecurity challenges.

— The Analyst: Hyposelenia